



西安交通大学

XI'AN JIAOTONG UNIVERSITY

本科毕业设计（论文）

面向项目级数据理解的图谱化 Text-to-SQL 智能体设计与实现

学院（部、中心）： 电信学部

专 业： 自动化

班 级： 自动化钱 2201（信息）

学生姓名： 陈必纯

学 号： 2222112595

指导教师： XXX

2025 年 06 月

（注：X 均为阿拉伯数字，如 2025 年 06 月）

摘 要

自然语言数据分析正在从“生成一条 SQL”转向“让智能体理解并持续维护一个数据项目”。在企业和科研场景中，关系数据库、表格文件、JSON 文档、项目说明、历史查询和业务规则往往共同构成分析上下文；真实问题也不只依赖表名和列名，还涉及隐式连接关系、领域术语、指标口径、样例值和历史经验。大语言模型具备良好的语言理解与代码生成能力，但在真实数据项目中仍面临上下文组织困难、schema/value grounding 不稳定、工具验证成本高以及知识难以跨问题复用等挑战。针对这些问题，本文设计并实现 Pontis：一种面向项目数据理解的图谱化智能体工作空间。Pontis 不以训练或微调专用 Text-to-SQL 模型为目标，而是在原始数据与大语言模型智能体之间构建可查询、可更新、可复用的项目知识层，使智能体能够围绕数据资产逐步检索、验证和积累上下文，再完成自然语言查询与数据分析任务。

本文提出以知识图谱为核心的项目工作空间架构，将文件、数据库、表、列、外键、统计摘要、语义描述、消歧规则和经验知识等统一建模为图实体及关系。系统通过自动化提取流程发布基础事实，并生成列统计、值分布、列重叠、外键验证、语义摘要和向量索引等派生知识；同时通过智能体探索沉淀难以完全规则化的连接经验、字段消歧和任务先验。在此基础上，Pontis 设计了面向数据分析的工具调用智能体，将图检索、元数据读取、SQL 查询、文本搜索和图写入等能力封装为工具，并通过轮次限制、探索检查、SQL 实体检查、桥接表检查和 SQL 消歧检查等护栏机制，降低模型在复杂数据库问答中的幻觉、错误连接和过度探索风险。

在 BIRD dev 全量 1534 道题的 protocol-level zero-shot 评测中，Pontis 取得 65.38% 的执行准确率，高于 Bash Agent 的 60.04%、CHESS 的 58.67%、Alpha-SQL 的 63.36%，并与 DeepEye-SQL 的 65.25% 基本持平。资源消耗方面，Pontis 的在线阶段平均每题调用 11.45 轮 LLM，其中大量输入来自可缓存的稳定项目上下文；若以 DeepSeek V4 Flash 的缓存输入、非缓存输入和输出价格为例作大致折算，其在线估算成本指数为每题 16132.8，加上预处理摊销后为每题 18709.1。实验结果表明，Pontis 的主要价值并不局限于提升单一基准精度，而是在准确率、可解释错误归因、上下文缓存和后续增量维护之间提供一种可分析的数据智能体系统结构。

关 键 词：数据智能体；Text-to-SQL；大语言模型；知识图谱

ABSTRACT

As data projects in enterprise and research scenarios continue to grow, relational databases, tabular files, JSON documents, and project documentation often coexist in loosely organized directories. Large language models have shown strong capabilities in natural language understanding and code generation, but they still struggle with real data projects because of limited context windows, implicit schema relationships, ambiguous domain terminology, and the lack of reusable analysis experience. To address these problems, this thesis designs and implements Pontis, a graph-native agent workspace for project data. Instead of training a specialized Text-to-SQL model, Pontis introduces a queryable, updatable, and reusable knowledge graph layer between raw data and LLM agents, enabling agents to understand a data project before answering natural language queries.

This thesis first presents a workspace architecture centered on Neo4j and Cypher. Files, databases, tables, columns, foreign keys, statistical summaries, semantic descriptions, disambiguation hints, and cross-project experience are modeled as graph entities and relationships. Source modules publish source-derived facts such as file-system and SQLite schema information before graph queries. Extractor modules generate derived knowledge including column statistics, value distributions, column overlaps, foreign-key validation, semantic summaries, and vector indexes. All upper-level tools and agents access the graph through a unified `workspace.cypher` interface. The thesis then presents a configurable tool-calling data agent. The full readonly profile includes tools such as `find`, `meta`, `query`, `grep`, `read`, `jd`, `bash`, `cypher`, and sub-agent calls, while the BIRD benchmark solving stage intentionally restricts the online tool set to `find`, `meta`, and `query`. Guardrails including round limitation, exploration checking, SQL entity checking, bridge-table checking, SQL disambiguation checking, value grounding, and final BIRD rule rechecking are introduced to reduce hallucination, incorrect joins, unsupported literal values, and unnecessary exploration.

Full evaluation on the BIRD development set contains 1,534 questions under a protocol-level zero-shot setting, where no method explicitly uses gold SQL from the BIRD training split. Pontis achieves 65.38% execution accuracy, compared with 60.04% for Bash Agent, 58.67% for CHESS, 63.36% for Alpha-SQL, and 65.25% for DeepEye-SQL. In terms of cost, Pontis uses 11.45 LLM rounds per query on average. Most of its input tokens come from cacheable project context. Using DeepSeek V4 Flash pricing as an illustrative price conversion for cached input, uncached input, and output tokens, Pontis has a runtime cost index of 16,132.8 per query and an amortized cost index of 18,709.1 per query after preprocessing. These results suggest that

ABSTRACT

the value of Pontis is not limited to improving a single benchmark score, but lies in providing a transparent, cacheable, and incrementally maintainable data-understanding workspace for general-purpose data agents.

KEY WORDS: large language model; agent; knowledge graph; Text-to-SQL; data workspace

目 录

1 绪论.....	1
1.1 研究背景与意义.....	1
1.2 主要贡献	2
1.3 论文结构	3
2 相关工作	4
2.1 Text-to-SQL 基准与真实场景演进	4
2.2 Schema Linking 与元数据增强	4
2.3 智能体式 Text-to-SQL 与执行验证	5
2.4 知识图谱与数据项目理解	5
2.5 评测噪声与系统指标	6
3 问题定义与总体框架	7
3.1 项目级数据理解任务定义	7
3.2 传统 Text-to-SQL 范式的局限	7
3.3 核心挑战与设计目标	8
3.4 图谱化工作空间总体框架	8
3.5 系统组成与数据流	9
3.6 图谱数据模型与统一接口	9
4 图谱化项目知识构建方法	11
4.1 节点身份、边与访问指针	11
4.2 自动化脚本式知识提取	12
4.3 列统计与值分布	13
4.4 智能体知识探索	14
4.5 语义摘要与向量索引	16
4.6 增量刷新与成本控制	17
5 工具智能体与执行验证机制	18
5.1 智能体循环	18
5.2 工具集合	18
5.3 提示组装	19
5.4 护栏机制	20
6 实验设计与结果分析	21
6.1 研究问题	21

6.2	数据集与评测指标	21
6.3	对比方法	22
6.4	主实验结果	23
6.5	预处理资源消耗	24
6.6	与流水线基线方法的差异	25
6.7	数据库结构与内容变动下的增量更新潜力	25
6.8	有效性威胁	26
7	基准局限与任务建模讨论	27
7.1	Text-to-SQL 任务建模局限	27
7.2	Bash Agent 反思结果的佐证	27
7.3	SQL 多重可达性	28
7.4	Query 语义欠定性	29
7.5	执行结果比对的局限	30
7.6	从静态评测到交互式业务目标	30
8	总结与展望	32
	致谢	33
	参考文献	34
	附录 A Pontis 代码模块说明	37

1 绪论

1.1 研究背景与意义

数据正在成为组织运行、科研发现和软件系统决策的核心基础设施。与早期主要围绕单一关系数据库或固定报表展开的数据访问不同，现代数据工作流通常横跨数据仓库、湖仓、业务语义层、BI 仪表盘、数据管道、实验记录、代码仓库和团队知识库。数据消费者也不再只满足于执行预定义 SQL，而是希望用自然语言提出探索性问题、追问分析过程、验证指标口径，并把得到的发现转化为后续决策或自动化动作。因此，数据系统面临的关键问题正在从“如何高效存储和查询数据”扩展为“如何让智能体理解、检索、验证并持续维护一个组织的数据知识”。

在这一背景下，数据智能体（data agent）成为学术界和工业界共同关注的方向。相比单次输入输出 Text-to-SQL 模型，数据智能体是能够围绕数据任务进行规划、调用工具、读取元数据、执行查询、验证结果并生成分析结论的系统。学术界的演进也体现了这一变化：Spider[1] 和 BIRD[2] 推动了 Text-to-SQL 从跨库泛化走向真实数据库和执行约束；Spider 2.0[3] 进一步把评测对象扩展到真实数据应用和企业级工作流；InsightBench[4] 与 Data Agent Benchmark[5] 则将问题推进到多步洞察生成、异构数据源和企业数据问答。由此可见，自然语言数据分析正在从“单条 SQL 是否生成正确”转向“智能体能否在复杂数据环境中稳定完成分析任务”。

工业界也在快速把数据智能体嵌入主流数据平台。Microsoft Fabric data agent 被定位为 Fabric 中的对话式分析组件，能够连接 OneLake 中的 lakehouse、warehouse、semantic model 和 KQL 数据库，并在权限与治理约束下选择合适的数据源和查询工具 [6]。Snowflake Cortex Agents 通过 Cortex Analyst 和 Cortex Search 在结构化与非结构化数据之间进行工具路由，使应用可以在企业数据治理范围内完成 SQL 分析和文档检索 [7]。Databricks Genie Space 则要求领域专家用数据集、示例查询和文本指南配置面向业务团队的自然语言分析空间，并允许随着数据和用户问题变化持续更新语义知识 [8]。Tableau Next 和 agentic analytics 的提出也反映了 BI 平台从可视化报表向“从数据到洞察再到行动”的智能体化分析体验转变 [9]。

这些趋势表明，真实的数据智能体系统必须同时处理四类能力：第一，能够在复杂数据资产中定位相关实体和上下文；第二，能够理解组织内部的业务术语、指标口径和历史经验；第三，能够通过工具在真实数据上验证假设，而不是只依赖模型参数中的常识；第四，能够在安全、权限和治理边界内运行，并为后续问题积累可复用知识。换言之，数据智能体的核心难点不只是 SQL 语法生成，而是上下文组织、语义 grounding、工具编排、评测可靠性和长期记忆的系统工程问题。企业内部系统的实践研究也显示，有效的 Text-to-SQL 系统需要索引数据库元数据、历史查询、文档和代码等多源知识，而不能只依赖表名和列名等简单元数据 [10, 11]。

因此，直接将大语言模型用于真实数据项目仍面临以下困难。

第一，数据库模式和项目上下文过大。真实数据库可能包含大量表、列、视图和项目文档。若将完整 schema、样例值和说明文档全部纳入提示词，会造成上下文冗长、成本上升和注意力分散；若预先过滤 schema，又可能错误丢弃关键表列。尽管当前大模型上下文窗口不断扩大，数据分析任务在数据库规模较小时可以直接输入完整数据库 schema；但当企业 schema 继续扩张时，系统仍需要可靠的上下文控制机制来降低模型认知复杂度 [12]。

第二，关系、语义和值口径常常是隐式的。许多数据库没有完整外键约束，表间连接依赖命名约定、值域重叠、历史查询或项目说明。列名相同并不意味着语义相同，列名不同也可能表达相同业务概念；即使表列选择正确，过滤值、枚举别名、日期边界和聚合口径也仍可能出错。相关研究表明，列统计、值分布、查询日志、业务规则和 LLM 生成的字段描述可以显著补足人工文档缺失的问题 [13][14][15]。

第三，模型必须在真实数据上验证假设。自然语言问题中的过滤值、枚举项、日期边界、比例分母和聚合粒度往往无法仅靠 schema 推断。SQL-PaLM、CHESS、ReFoRCE 和 APEX-SQL 等系统分别通过执行反馈、候选选择、多智能体协作和列探索来提升可靠性 [16][17][18][19]。这些工作说明，Text-to-SQL 正在从一次性生成转向带工具调用、检索、验证和修正的 test-time framework。

第四，评测结果本身需要谨慎解释。BIRD 等基准中存在自然语言歧义、gold SQL 口径差异和标注错误；已有研究发现，噪声和标注问题会影响模型排名和方法判断 [20, 21]。这意味着系统评估不能只报告最终执行准确率，还应分析错误来源、可缓存上下文、运行时 token 成本和 LLM 调用轮次。

本文认为，真实数据项目中的自然语言分析不应只被视为单次 SQL 生成问题，而应被视为“智能体如何持续理解并维护一个数据项目”的问题。Pontis 因此在原始数据和 LLM 智能体之间引入图谱化工作空间，将 schema、统计、关系、摘要、消歧和经验统一表示为可查询知识，使智能体能够通过工具逐步获取精确上下文，而不是每道题从零扫描整个项目。

1.2 主要贡献

围绕上述问题，本文设计并实现了 Pontis 图谱化数据工作空间。本文的主要贡献如下：

(1) 提出一种图谱化数据工作空间架构。该架构以 Neo4j 为持久化图存储，以 Cypher 为统一查询接口，以自动化脚本和智能体探索共同维护项目知识，避免在每次问答中直接拼接全部数据上下文。

(2) 实现一套面向数据库项目的知识提取与组织机制。系统能够自动暴露文件系统、SQLite 模式、表列结构和外键关系，并进一步生成列级统计、值样例、列重叠、AI 摘要和语义向量等派生知识。

(3) 设计了结合工具调用和护栏的 LLM 数据智能体。系统将图谱检索、元数据查看、SQL 执行、文件搜索和图写入操作封装为工具，并通过 SQL 实体检查、桥接表检

查和消歧检查减少常见数据库问答错误。

(4) 对 Pontis 与代表性基线方法进行统一实验比较和系统指标分析。本文将 Pontis 与 Bash Agent、CHESS、Alpha-SQL、DeepEye-SQL 等方法放在统一评测协议下比较, 除准确率外统计 token 消耗、LLM 调用轮次等效率指标, 并结合错误案例讨论静态基准的解释边界, 使系统型贡献能够被量化分析。

1.3 论文结构

本文结构如下: 第二章介绍相关工作; 第三章定义项目级数据理解问题并给出图谱化 Text-to-SQL 智能体的总体框架; 第四章阐述离线或预处理阶段的图谱化项目知识构建方法; 第五章介绍在线问答阶段的工具智能体与执行验证机制; 第六章给出实验设计、主实验结果、基线对比、预处理成本、增量更新潜力和有效性威胁; 第七章讨论当前 Text-to-SQL 基准的评测局限与任务建模问题; 第八章总结全文并展望未来工作。

2 相关工作

2.1 Text-to-SQL 基准与真实场景演进

Text-to-SQL 是自然语言处理与数据库系统交叉领域的重要任务，其目标是将自然语言问题转换为可执行 SQL。早期方法多依赖规则和语义解析，随后 Seq2SQL[22] 等方法将神经网络和强化学习引入 SQL 生成。Spider[1] 的提出推动了跨数据库泛化研究，RAT-SQL[23] 通过关系感知编码器建模问题与数据库 schema 之间的关系，代表了在深度模型时代显式建模 schema linking 的典型路线。

大语言模型出现后，Text-to-SQL 的研究重心从模型结构设计逐渐转向提示工程、检索增强、任务分解和执行反馈。C3[24] 和 DAIL-SQL[25] 系统比较了提示表示、示例选择和自修正等因素，DIN-SQL[26] 则将复杂 SQL 生成拆分为 schema linking、查询分类和 SQL 生成等子任务。这些工作表明，强 LLM 不一定需要任务级微调才能完成复杂 SQL，但其效果高度依赖上下文组织和推理流程设计。BIRD[2] 进一步引入更大的数据库、外部 evidence、真实值理解和 SQL 效率要求，使 Text-to-SQL 从语法映射转向数据库内容理解。

近年的基准进一步强调企业级复杂性和端到端数据分析能力：一类工作将任务扩展到真实数据应用、企业级 schema、查询日志和 SQL workflow，另一类工作则把评测对象从单条 SQL 扩展到多步业务洞察生成 [3][27][4]。这些工作共同说明，真实 Text-to-SQL 系统需要处理远大于教学数据集的 schema 空间、业务术语、私有规则、SQL 方言和多轮分析流程。本文的 Pontis 研究也基于这一趋势，将 Text-to-SQL 问题从单次生成任务拓展到“项目级数据理解”。

2.2 Schema Linking 与元数据增强

Schema linking 负责将自然语言问题中的实体、属性和约束映射到数据库表列，是 Text-to-SQL 系统的核心环节。传统方法通常将 schema linking 作为 SQL 生成前的过滤步骤，但近期研究对这一环节提出了更细的分析。Maamari 等人的研究指出，当完整 schema 能放入上下文且模型推理能力足够强时，过早过滤 schema 可能因为召回错误而伤害生成结果 [12]。这一结论说明，schema linking 不宜被理解为“越早过滤越好”，其价值取决于 schema 规模、上下文窗口、模型能力和召回风险。对 Pontis 而言，关键不仅是裁剪 schema，而是把可复用的上下文组织为可查询的项目知识，使智能体在需要时逐步获取证据。

单纯依赖表名和列名也不足以解决真实场景中的语义问题。自动元数据抽取研究从数据库 profiling、列级统计、值样例、历史查询日志和 LLM 生成字段摘要中提取补充信息 [13]；schema refinement 工作则把原始数据库整理为更易解释的语义层，降低用

户问题与底层 schema 之间的语义距离 [15]。面向多表问答的 schema graph 方法进一步说明，表列之间的连接关系、候选路径和人工确认信息适合用图结构表达，而不应完全依赖模型在生成时临场推断 [14]。

除此之外，schema linking 只解决“选择哪些表列”的问题，并不能自动保证过滤值、枚举映射、日期边界和聚合口径正确。BIRD 的设计已经强调数据库值和外部 evidence 的作用 [2]，而企业系统中常见的业务简称、历史口径和隐式指标定义则使 value grounding 更加困难。RubikSQL 将 NL2SQL 视为长期学习任务，持续维护业务术语、规则和失败经验 [28]；LinkedIn 的企业 Text-to-SQL 系统也把元数据、查询日志、文档和代码共同纳入知识图谱 [10]。这些工作表明，真实数据项目中的上下文不只是 schema 文本，而是 schema、值、关系和经验的组合。

Pontis 与这些工作的共同点在于重视元数据、值分布和关系知识；不同点在于，Pontis 将这些信息统一落到项目级图谱工作空间中，并通过工具接口让智能体按需读取，而不是一次性把全部增强 schema 拼入 prompt。

2.3 智能体式 Text-to-SQL 与执行验证

工具调用是增强 LLM 能力的重要途径。在 Text-to-SQL 中，工具调用可以弥补模型静态知识不足的问题：模型可以执行 SQL 验证假设，通过搜索定位字段，通过样例值确认枚举，通过错误反馈修正语法和逻辑。

围绕 BIRD 和 Spider 2.0，已有方法逐渐形成多阶段、可验证的 test-time framework。SQL-PaLM[16] 研究了 few-shot prompting、执行过滤、consistency decoding、指令微调 and 数据库内容使用对性能的影响；CHESS[17] 使用信息检索、schema 选择、候选生成和单元测试等多个 agent 分工协作；ReFoRCE[18] 引入表压缩、格式约束、自修正、共识选择和列探索；APEX-SQL[19] 将生成过程组织为假设验证循环，通过真实数据探索落地语义。上述工作说明，复杂 Text-to-SQL 的关键并不只是更强的单次生成，而是如何组织检索、探索、验证和选择。

除显式工具调用外，候选集选择和 test-time scaling 也是提升稳定性的常见路径。self-consistency[29] 说明多条推理路径的一致性可以提高复杂推理可靠性，Text-to-SQL 系统也常借助多候选生成、执行筛选和一致性选择来降低偶然错误。这类方法提升了复杂任务上的准确率，但通常也会增加 token 消耗和 LLM 调用轮次，因此需要和统一实验指标一起分析。

Pontis 也采用工具智能体范式，但其重点不是构造更多候选 SQL，而是提供一个可持久化的数据理解层。智能体的每次检索、元数据查看和 SQL 验证都围绕图实体展开，后续问题可以复用已有的统计、摘要、消歧和经验知识。

2.4 知识图谱与数据项目理解

知识图谱将实体及其关系表示为图结构，适合表达数据库 schema、文件组织、表列归属、外键关系和业务概念之间的联系。传统知识图谱构建强调从文本中抽取实体

和关系，而数据项目场景中的图谱构建还需要处理结构化源事实和统计派生知识。企业 Text-to-SQL 实践已经显示，知识图谱可以把数据库元数据、历史查询、文档和代码等多源信息组织为可检索上下文，为 SQL 生成和错误修复提供依据 [10]。

在企业数据分析场景中，知识图谱的价值不仅在于存储概念关系，还在于为智能体提供可操作的项目地图。Schema graph 相关工作使用图结构减少模型对隐式连接关系的猜测 [14]；RubikSQL 将长期运行中沉淀的业务知识、错误修正和查询经验组织为可复用知识库 [28]；LinkedIn 的企业系统使用知识图谱统一数据资产、元数据和查询经验 [10]；Uber QueryGPT 则结合向量数据库和相似查询检索，将自然语言问题和内部数据模型知识连接起来 [11]。这些实践说明，Text-to-SQL 系统需要一种位于数据库和模型之间的中间表示，用来承接不断变化的项目知识。

Pontis 的图谱并不是静态业务知识库，而是围绕项目数据持续生成和更新的工作空间。文件系统和数据库 schema 可以作为源事实按需刷新，AI 摘要、列统计、列重叠和人工经验可以作为派生事实持久化。通过这种方式，Pontis 将“当前项目是什么”和“过去分析学到了什么”统一到一个可查询图中。与只服务于单次 SQL 生成的增强 prompt 相比，图谱工作空间更适合跨问题复用、错误归因和后续知识更新。

2.5 评测噪声与系统指标

Text-to-SQL 评测通常依赖执行准确率，即比较预测 SQL 与参考 SQL 的执行结果是否一致。然而，执行结果并不总能完整反映语义正确性：不同 SQL 可能因输出列、排序、聚合口径或边界条件差异而产生不同结果；模型也可能为了可读性或可解释性生成比参考 SQL 包含更多辅助信息的结果，却因为输出格式不一致而被判错；参考 SQL 本身也可能存在标注错误或问题歧义。针对 BIRD 的噪声分析指出，问题和 gold SQL 中的错误会影响模型表现和结论可靠性 [20]；进一步的 benchmark 审计研究也表明，标注错误可能显著改变模型排名 [21]。

随着基准复杂度提高，评测对象也从“单条 SQL 是否匹配”扩展为“系统是否能稳定完成分析任务”。因此，系统比较需要同时报告错误类型和资源消耗，尤其要区分 schema linking 失败、value grounding 失败、SQL 语义错误、输出口径差异和评测噪声。

基于这一考虑，本文在评估 Pontis 时不仅关注执行准确率，还关注 `llm_rounds`、`cached_input_tokens`、`uncached_input_tokens`、`output_tokens` 和 `total_tokens`。这些指标更适合衡量图谱化工作空间的价值：即它是否能把可缓存的稳定上下文和每题运行时新增上下文区分开，减少重复输入，并提高 schema/value grounding 的可靠性。

此外，真实数据分析系统中的查询通常并非完全孤立。用户会围绕同一数据库持续提出相关问题，历史正确 SQL、失败案例、字段消歧和业务口径都可能影响后续回答。本文的主实验仍遵循 BIRD 的逐题独立评测协议，不在测试过程中利用前一题答案改造后一题；但从系统设计角度看，Pontis 将项目知识建模为可更新工作空间，为未来研究 query-log 式连续评测和经验沉淀提供了接口基础。

3 问题定义与总体框架

前两章已经说明，真实数据分析任务正在从单次 SQL 生成转向项目级数据理解。本章进一步给出本文的任务定义、传统 Text-to-SQL 范式的局限、系统设计目标，以及 Pontis 所采用的图谱化工作空间总体框架。通过先界定问题和设计目标，再介绍系统组成与数据流，可以为后续章节的具体实现提供清晰基础。

3.1 项目级数据理解任务定义

本文关注的不是孤立的 SQL 语法生成，而是项目级数据理解（project-level data understanding）任务。给定一个数据项目 P 、一个自然语言问题 q 和一组可用工具 T ，系统需要理解项目中的数据库、表、列、文件、统计信息、业务术语和潜在关系，并生成能够回答用户问题的 SQL 或分析结果。该过程可以抽象为：

$$g(q, P, T, K) \rightarrow (sql, r), \quad (3-1)$$

其中 K 表示系统在项目中已经构建或沉淀的知识， sql 表示最终生成的查询语句， r 表示执行结果或分析回答。与传统 Text-to-SQL 中常见的 $f(query, database) \rightarrow sql$ 不同，项目级数据理解强调三点：第一，输入不只是数据库 schema，还包括文件、说明文档、统计画像和历史经验；第二，系统可以通过工具逐步检索和验证，而不是一次性输出 SQL；第三，项目知识可以在不同问题之间复用和更新。

在本文的 BIRD 实验中， P 主要表现为一个 SQLite 数据库及其问题 evidence；在更一般的数据项目中， P 还可以包含表格文件、JSON 文件、项目文档和业务规则。Pontis 的目标是在这些异构项目资产之上建立一个统一的工作空间，使智能体面对具体问题时能够先定位相关数据对象，再检查真实值和连接关系，最后生成并验证 SQL。

3.2 传统 Text-to-SQL 范式的局限

传统 Text-to-SQL 通常把任务建模为从自然语言 query 和 database 到 SQL 的单次映射。该范式适合描述 benchmark 中的逐题评测流程，但在真实数据项目中存在明显局限。

第一，单句 query 往往不足以唯一确定 SQL。自然语言问题通常依赖用户对数据库结构和业务口径的先验理解，但这些先验并不会完整写入 query。当同一术语可以映射到多个表列，或者同一指标存在多个聚合粒度时，模型只能在若干可能解释中选择一个。此时错误不一定来自 SQL 语法能力不足，而可能来自问题本身的欠定性。

第二，同一业务目标在 SQL 层面具有多重可达性。为了回答同一个问题，可能存

在多种语义相近甚至对真实用户更有用的 SQL 写法。基准的 gold SQL 只能代表其中一种标注选择，执行结果比对也会受到输出列、排序、格式、数值类型和聚合粒度的影响。因此，执行准确率是必要指标，但不能完全代表系统是否真正满足业务目标。

第三，数据库理解不能只依赖表名和列名。真实项目中的连接关系、枚举值、日期边界、字段口径和业务简称往往隐藏在数据值、文档或历史经验中。若每道题都从零探索这些信息，系统成本高且不稳定；若完全不探索，又容易产生 schema linking、value grounding 和 join path 错误。

3.3 核心挑战与设计目标

基于上述问题，本文将图谱化 Text-to-SQL 智能体的核心挑战归纳为四类。

(1) 上下文定位。系统需要在复杂项目中找到与当前问题相关的数据库、表、列、文件和知识实体，同时避免把无关 schema 全部塞入 prompt。

(2) 语义 grounding。系统需要利用真实数据值、列统计、样例值和文档说明，把自然语言中的实体、枚举、时间范围和指标口径落到具体表列。

(3) 执行验证。系统需要通过 SQL 执行、结果检查和错误反馈验证假设，降低一次性生成 SQL 的偶然错误。

(4) 知识复用与更新。系统需要把可重复使用的统计、摘要、连接关系和消歧经验保存下来，并在数据库结构或内容变化时支持局部刷新。

Pontis 的设计目标因此不是替代所有专用 Text-to-SQL pipeline，而是在原始数据和 LLM agent 之间提供一个可查询、可更新、可复用的项目知识层。该知识层既要支持离线提取，也要支持在线问答；既要能被 agent 读取，也要能记录来源、身份和后续更新线索。

3.4 图谱化工作空间总体框架

Pontis 的核心思想是在原始数据源和智能体之间建立图谱化中间层。系统整体由四个部分组成：原始数据源、图谱工作空间、知识提取器和工具智能体。

原始数据源包括 SQLite 数据库、表格文件、文本文件和项目说明文档。图谱工作空间负责把这些数据资产表示为带有稳定身份的节点与关系。知识提取器在离线或预处理阶段生成基础事实、统计画像、语义摘要、关系候选和向量索引。工具智能体则在在线阶段围绕用户问题调用图检索、元数据读取、SQL 查询和文本搜索等工具，逐步构造和验证最终 SQL。

从具体工作流程看，一个数据库进入 Pontis 后，系统首先发布数据库、表、列和外键等源事实；随后计算列统计、样例值、列重叠、语义摘要和向量；再由智能体探索模块补充难以完全规则化的 schema 摘要、关系线索、局部提示和 README 知识。在线问答时，query agent 先根据用户问题检索相关实体，读取表列元数据和 README，再用 SQL 查询工具检查样例值或验证中间假设，最后输出 SQL。若最终 SQL 引用了未核实的实体、可能遗漏桥接表、包含数据库中无法命中的字面值，或触发局部消歧规

则，护栏会阻断输出并要求 agent 继续检查。

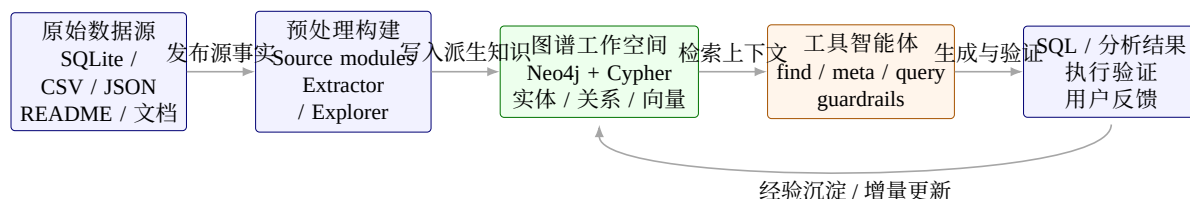


图 3-1 Pontis 图谱化工作空间总体框架

3.5 系统组成与数据流

Pontis 首先通过确定性的自动化脚本将数据源中的基础事实发布为图节点和边，例如目录、文件、数据库、表、列和外键；随后继续在图谱之上生成统计、摘要、关系和向量等增强信息。Neo4j 图谱工作空间负责持久化这些事实和派生知识，并提供 Cypher 查询接口。工具智能体通过工具访问图谱和原始数据，逐步完成自然语言分析任务。

系统的核心接口边界是 `workspace.cypher(...)`。上层工具和知识提取流程均通过该接口访问图谱，避免直接依赖底层图存储的具体实现。该设计将数据源刷新、图查询和结果指针解析逻辑集中在工作空间层，从而降低工具层与存储层之间的耦合。

3.6 图谱数据模型与统一接口

Pontis 将项目数据表示为属性图。在典型的数据库 Text-to-SQL 场景中，主要涉及的实体类型包括：

- (1) 文件实体。表示项目目录中的文件，包含路径、文件类型和源访问指针等属性。
- (2) 数据库实体。表示 SQLite 等数据库文件，作为表、视图和列的上级节点。
- (3) 表与视图实体。表示数据库表、视图或表格文件中的逻辑表，记录名称、来源和结构信息。
- (4) 列实体。表示表中的字段，记录列名、类型、样例值、统计信息和语义描述。
- (5) 外键和关系实体。表示显式外键、推断关系、列重叠和消歧知识。
- (6) 知识实体。表示人工或自动沉淀的规则、经验、错误分析结果和基准案例。

Pontis 使用 Neo4j 作为图存储后端。工作空间层对外暴露 Cypher 接口，基础事实发布脚本和工具通过 Cypher 语句完成图实体写入、匹配和查询。选择 Neo4j 的主要原因包括：

- (1) Cypher 能直接表达图模式匹配、邻居遍历和多跳关系查询，适合表列关系、外键关系和知识实体检索。
- (2) 图存储与工具层解耦。工具只需要关心实体 ref、标签和查询结果，不需要自己维护图索引。
- (3) 跨项目路由更清晰。Pontis 可以为不同项目配置不同 Neo4j 端口或数据库，使同一套工具能够访问多个项目图。

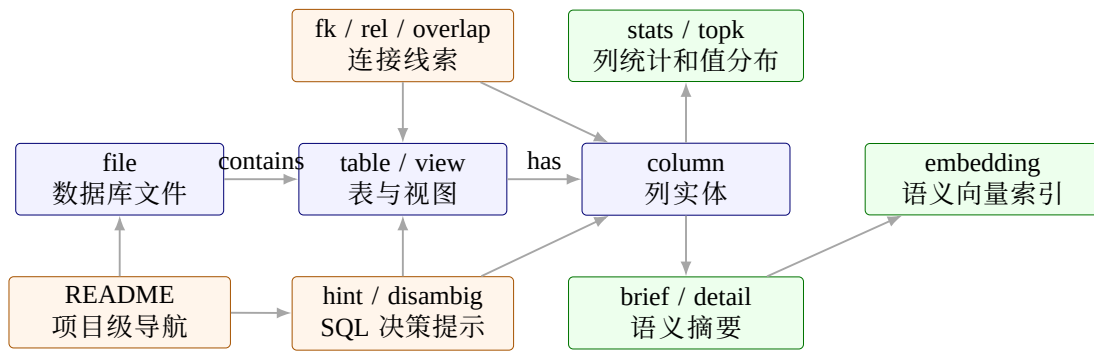


图 3-2 Pontis 图谱数据模型示意

(4) 后续可利用 Neo4j 的索引、向量检索和查询优化能力，支撑更大规模数据项目。

4 图谱化项目知识构建方法

第四章介绍离线或预处理阶段的图谱化项目知识构建方法。面向真实数据项目的自然语言分析任务，系统需要处理的上下文并不限于数据库 schema。除表名、列名和外键等结构化事实外，列值分布、字段语义、隐式连接关系、业务术语、历史经验和非结构化说明文档都会影响最终 SQL 的正确性。传统 Text-to-SQL 方法通常在推理时临时拼接 schema 文本，而企业数据和真实基准中常见的弱文档、弱约束和多源数据使这种方式难以稳定覆盖必要证据。因此，Pontis 将项目知识组织为可增量更新的图谱工作空间，使工具智能体能够以实体和关系为单位读取上下文，而不必在每次问答时重新扫描完整项目。

现有研究将空值率、不同值数量、数据类型、值模式、函数依赖和包含依赖等信息视为理解数据集的重要元数据 [30]；Aurum 等数据发现系统通过图结构组织企业数据资产及其关系，以支持跨数据源发现和查询 [31]；自动元数据抽取和 schema graph 相关工作则进一步说明，列统计、字段描述、连接路径和人工确认知识能够补足原始 schema 的语义缺口 [13][14]。Pontis 的目标不是单独复现其中某一种技术，而是把这些知识类型纳入同一运行时工作空间，为后续的检索、探索、验证和经验复用提供基础。

Pontis 的图谱构建采用分阶段、可增量的组织方式。首先，自动化脚本把文件系统、文本文件、表格文件和数据库模式转换为图节点与边，形成稳定的源事实视图；其次，知识提取流程在源事实之上补充列统计、值分布、结构模式、关系候选、语义摘要和向量索引；最后，智能体探索流程针对难以完全规则化的语义关系和消歧问题生成经验性知识。三类结果共享同一图谱存储，但在来源、生成方式和可信度上保持区分。确定性事实和统计画像主要来自脚本，语义判断和连接经验主要来自智能体探索，从而降低错误推断对基础事实的污染风险。

4.1 节点身份、边与访问指针

Pontis 在图谱中同时维护“实体身份”和“原始数据访问能力”。实体身份由路径、引用和内部 id 共同保证：文件节点通常以路径合并，表和列节点使用数据库路径、表名和列名组成稳定引用，外键和关系实体则通过两端表列确定。所有节点都会获得存储层生成的内部 id，用于稳定边连接和工具显示。稳定身份是图谱复用的前提：如果同一张表、同一列或同一段文本在不同运行中无法被识别为同一实体，统计结果、摘要、向量和智能体经验就难以增量更新，也无法可靠地被后续问题复用。

图谱建模遵循一个基本原则：具有独立语义或需要被检索、更新、引用的内容均建模为节点，例如表、列、外键、业务术语、历史经验和错误分析结果；边主要承担实体之间的连接与导航作用，复杂语义则保留在节点属性或关系实体节点中，便于 agent 直接探索和查询。

从数据发现角度看，图谱中的边主要作为被智能体利用的导航路径。文件到文本 chunk、数据库到表、表到列、列到统计、列到候选关系、实体到摘要和向量索引之间的连接，使系统能够从一个自然语言查询逐步缩小到相关数据源、相关表列和可能的连接路径。Schema graph 工作表明，显式维护表列之间的图结构能够减少模型在多表问答中对隐式关系的临场猜测 [14]。Pontis 将这一思想扩展到项目级数据资产，使关系型表、半结构化文件和智能体生成的经验知识可以处在同一个查询空间中。

4.2 自动化脚本式知识提取

自动化脚本式知识提取负责生成可以由数据源稳定推导或批量计算的图谱知识。每个提取单元接收一个 workspace 对象，通过 Cypher 和工具函数读取源事实，并将结果写回图谱。其任务既包括提取文件、文本、表格、数据库、表、列和外键等基础事实，也包括在基础事实之上生成统计、摘要、关系候选和向量索引。因此，这类脚本可以离线批量运行，也可以按实验需要选择性执行。

该部分的核心价值在于把大量低层、重复、可计算的观察过程从在线智能体循环中前移。对于一个新项目，智能体如果在回答每个问题时都临时查看目录、浏览 schema、执行 SELECT DISTINCT、采样字段值并判断候选关系，会造成较高的 LLM 轮次和 token 消耗。自动化提取将这些步骤转化为可缓存的图谱事实，使在线阶段更多地围绕已提取证据进行推理。数据画像和自动元数据抽取研究均表明，这类统计性和描述性元数据能够显著改善数据理解过程 [30][13]。

系统中的主要提取单元如表 4-1 所示。这里的“提取单元”既包括确定性脚本，也包括在预处理阶段运行的专门 explorer；二者都通过同一工作空间接口写回图谱，但来源和可信度不同。

表 4-1 Pontis 主要知识提取单元

提取单元	功能
源事实发布模块	提取目录、文件、文本、SQLite 数据库、表、视图、列和外键等基础实体
表格与 JSON 结构模块	为 CSV/TSV/JSON 文件生成可检索结构、字段样例和模式摘要
数据库列统计模块	提取 SQLite 列统计、样例值、高频值、近似基数和空值比例
外键验证模块	验证数据库声明外键的有效性，并记录覆盖率和异常情况
列值重叠发现模块	计算列值重叠并生成潜在连接关系候选
列语义摘要模块	使用 LLM 根据列名、类型、统计和值样例生成数据库列摘要
schema_prepare	在预处理阶段为表列写入摘要，并确认高置信度关系线索
entity_hints	生成 row grain、字段落点、join 后果、格式风险等 SQL 决策提示
readme	将已生成的表列摘要、关系和风险压缩为库级 README
query_overview	在允许 transductive 设置时，从同库问题和 evidence 中抽取查询先验
语义向量模块	为存在 detail 的实体生成 embedding，并维护向量索引

自动化提取框架具有三个设计特征。第一，提取单元粒度较细，可以根据实验需要选择只运行源事实、统计、摘要、explorer 或语义向量模块。第二，执行顺序可配置，

既可以先运行基础结构提取和轻量统计，再运行关系发现与 LLM 摘要，也可以在基准实验中跳过高成本模块。第三，提取结果回写到同一个图谱工作空间中，后续工具和智能体无需感知提取过程的内部实现，只需查询图实体。

本文 BIRD dev 实验中采用的是受控预处理流水线，而不是启用 Pontis 中所有 extractor。实际默认流程包括四组：静态模块 `db_column_stats_approx`、`db_fk_validate`、`db_column_overlap`；LLM 列摘要模块 `ai_db_column_summary`；agent 模块 `agent_schema_prepare`、`agent_entity_hints`、`agent_readme`；以及 `semantic_embedding`。而 `agent_join_detect`、`agent_disambiguate`、`agent_query_overview`、文本分片和 CSV/JSON 摘要等属于通用或可选模块，并非本次主实验默认路径。这样区分可以避免把系统潜在能力与实际实验配置混为一谈。

在典型数据库项目中，自动化流水线可以分为四个阶段。第一阶段读取表、列和外键等基础节点，建立基础图；第二阶段扫描真实数据，生成列统计、样例值和高频值；第三阶段根据外键、值域重叠和统计特征生成关系候选；第四阶段使用 LLM 或 embedding 服务生成语义摘要和向量索引。各阶段的输出都以节点属性或派生实体的形式写回图谱，因此后续阶段可以复用前序阶段的结果。

4.3 列统计与值分布

列统计是 Text-to-SQL 中的重要上下文。仅靠列名往往无法判断一个字段的真实含义，例如 `type`、`id`、`status` 等列名在不同表中可能含义不同；相反，字段中的真实取值、空值比例、基数和分布形态往往能揭示其业务角色。数据画像研究将这类信息归入单列元数据，并进一步讨论唯一列组合、函数依赖和包含依赖等跨列特征 [30]。Pontis 在系统实现上先聚焦对在线问答最直接有用的轻量统计，对数据库和表格文件中的列提取以下信息：

- (1) 基本类型信息，包括 SQLite 声明类型和归一化类型。
- (2) 空值、不同值数量、样例值和文本长度等轻量统计。
- (3) 数值列的范围和分布摘要。
- (4) 文本列的高频值，用于帮助模型理解枚举字段和业务取值。

这些统计信息会写入列实体的元数据，并在 `meta` 工具中呈现。与直接把整列数据交给 LLM 不同，统计摘要将大量原始值压缩成稳定、低成本上下文。对于大列，系统不要求保存完整 `distinct` 集合，而是使用近似基数估计和流式高频项统计来降低预处理成本；对于文本列，系统保留少量去重样例和 `top-k` 高频值，使智能体能够判断枚举值、编码格式和自然语言问题中的值映射。

列统计同时服务于性能和准确率。性能上，它避免智能体为了理解一个枚举列而反复执行 `SELECT DISTINCT`；准确率上，它帮助模型判断过滤条件是否需要大小写转换、是否存在缩写、是否应该使用编码列还是描述列。许多 `value grounding` 错误并非源于表列召回失败，而是源于模型缺乏列中真实取值的证据。APEX-SQL 也强调通过真实数据探索验证列角色和查询假设 [19]。Pontis 将这类证据预先压缩到图谱中，使在线智能体可以用一次元数据读取获得候选值、分布和异常线索，从而降低 SQL 探索轮次。

列统计还为后续语义摘要和关系发现提供输入。列摘要模块需要依赖样例值、高频值和基数判断字段是否为枚举、标识符、金额、日期或自然语言描述；列值重叠模块则需要利用 `distinct` 值和覆盖率判断两列是否可能构成连接路径。因此，统计画像并不是孤立的辅助信息，而是 Pontis 图谱中从原始数据到高层语义知识的中间层。

4.4 智能体知识探索

自动化脚本适合生成稳定事实和统计画像，但真实数据项目中仍有一类知识难以完全规则化：字段语义需要结合上下文解释，隐式连接关系需要在全局 schema 中验证，同名列和近义字段需要根据样例值区分，长文档和说明材料中的业务口径也需要被压缩成可执行规则。Pontis 将承担这类任务的预处理智能体称为 `explorer`。与最终回答用户问题的 `query agent` 不同，`explorer` 的目标不是直接生成 SQL，而是在离线或半离线阶段把可复用的数据理解写回图谱。

`Explorer` 的设计并不是把一个通用 `agent` 放到项目中自由探索，而是采用专门化任务、受限工具和保守写入的组合。`Toolformer` 和 `ReAct` 说明了语言模型可以通过工具调用在外部环境中获取信息并逐步推理 [32][33]；`CHESS`、`ReFoRCE` 和 `APEX-SQL` 进一步把 `Text-to-SQL` 拆解为信息检索、schema 选择、列探索、候选生成和验证等子过程 [17][18][19]。Pontis 的 `explorer` 采用类似的任务分解思想，但输出对象不是一次性 SQL 候选，而是图谱中的长期知识。系统为 `explorer` 显式指定 `prompt`、工具集合和写入范围，通常只启用与任务相关的 `find`、`meta`、`query`、`update_meta`、`create_entity` 和 `add_edge` 等工具，并通过轮次护栏限制无界探索。

系统中的主要 `explorer` 如表 4-2 所示。它们按照知识类型分工，而不是按照代码目录简单分类：摘要类 `explorer` 负责把表、列、文件和结构模式转化为自然语言语义；关系类 `explorer` 负责把候选连接线索提升为高置信度关系；提示类 `explorer` 负责记录会直接改变 SQL 选择的字段、值、粒度和 `join` 决策。

其中，关系知识体现了 Pontis 对图谱写入的保守态度。真实数据库中并非所有连接关系都有显式外键，尤其在 `benchmark` 数据库、历史业务库和数据湖表格中，连接规则常常只存在于命名约定、值域重叠或业务经验中。数据发现系统通常需要在大量数据资产之间发现相关表、可连接列和可联合数据集 [31][34]；`schema graph` 方法也说明，多表问答需要显式维护可解释的连接路径，而不能完全依赖 LLM 在生成时猜测 [14]。Pontis 因此同时利用三类信息发现关系：数据库声明外键、列值重叠候选和智能体验证后的 `rel` 实体。列值重叠模块批量生成候选证据，`schema_prepare` 或可选的 `join_detect` 再读取外键、`overlap`、列统计和样例值，检查是否已有确定关系覆盖、是否存在传递冗余、是否与全局结构冲突，只有在高置信度下才写入 `rel`。这种“候选生成 + 保守确认”的设计接近 `APEX-SQL` 中用真实数据探索验证假设的思路 [19]，但 Pontis 将验证结果固化为可复用图谱知识。

`entity_hints` 和可选的 `disambiguate` 处理的是另一类高频错误：字段名相同不代表语义相同，字段名不同也可能指向同一业务概念。例如多个表中都存在 `id`、`name` 或 `type` 时，最终 SQL 生成前需要确认字段来源；而 `Free Meal Count` 与 `FRPM Count`

表 4-2 Pontis 主要 explorer 的设计职责

Explorer	设计思路	写入的知识
schema_prepare	面向数据库结构的预处理。先为表列补充摘要,再基于外键、overlap、样例值和表职责确认高置信度连接线索。	表、列、数据库、外键和关系实体的 brief/detail, 以及必要的 rel 实体。
entity_hints	面向 SQL 决策的提示抽取。重点维护 row grain、field landing、fact table selection、join consequence、aggregation grain 和 format risk。	基础实体的 hints 属性, 以及邻接的 hint/disambig 实体。
readme	项目级知识压缩。复用已生成的表列摘要、关系、提示和数据质量线索, 形成面向后续 agent 的库级导航文档。	README 节点的项目概览、主要对象、关系结构、数据质量风险和建议探索路径。
query_overview	面向 batch 或 transductive 场景的查询先验抽取。读取同库问题和 evidence, 压缩出字段口径、输出形状和候选消歧规则。	README 的 query-derived 章节, 以及必要的 knowledge/disambig/rel 实体。
analyze/join_detect/disambiguate	通用可选 explorer。分别承担全库理解、独立连接发现和独立消歧审计, 适合在非 BIRD 或更重预处理场景中启用。	表列摘要、rel、disambig 和相关补充说明。
text_chunk/json_pattern/csv_summary	面向半结构化和非结构化文件的语义整理。	文本 chunk、JSON 模式摘要、CSV 字段摘要和文件级说明。

这类近义字段也可能存在包含关系或统计口径差异。相关 explorer 会按列名、近名模式和 SQL 决策风险收集候选实体，再通过列统计、样例值、表职责和已有关系判断是否真的存在歧义。确认后，系统创建 disambig/hint 实体，并把短结论写入基础实体 hints，后续 SQL 消歧护栏会在高风险输出时提醒 query agent 读取这些信息。Agentic schema refinement 强调通过语义层整理降低用户问题与底层 schema 的距离 [15]；Pontis 的提示和消歧 explorer 可以看作面向局部歧义的轻量 schema refinement。

schema_prepare、readme 和可选的 analyze/text_chunk 则负责把数据资产转化为可检索、可复用的语义描述。schema_prepare 并非机械遍历所有实体，而是按数据库入口、核心表、依赖表、列统计和关系实体的顺序建立理解，并优先读取精确属性，以避免一次性展开全项目导致上下文膨胀；文本 explorer 则为长文档创建语义 chunk。这些设计对应于数据画像、语义类型检测和数据湖发现研究中的共同观点：数据理解需要结合结构、统计、值模式和上下文表示，而不是只依赖文件名或字段名 [30][35][34]。

readme 体现了 Pontis 对长期知识的利用。其把已有摘要、关系、消歧和数据质量线索压缩成库级 README，使后续 agent 第一次进入项目时不必从零开始。RubikSQL 将工业 NL2SQL 视为长期学习任务，强调通过数据库画像、规则挖掘和知识库维护持续积累领域知识 [28]；LinkedIn 和 Uber 的企业实践也显示，文档、业务规则和知识图谱能够改善自然语言到 SQL 的稳定性 [10][11]。

4.5 语义摘要与向量索引

Pontis 中的 brief 和 detail 是面向智能体的核心文本属性。列统计提供字段取值层面的证据，AI 摘要进一步补充字段或表的业务语义。系统会对表、列和文本文件生成摘要，并通过语义向量模块为 detail 文本生成向量。语义类型检测和表理解研究表明，字段含义通常需要结合列名、值分布、字符模式和表上下文判断，而不能只依赖表头字符串 [35]。因此，Pontis 的摘要生成也不是简单改写列名，而是把统计画像、样例值、上下游关系和文件上下文压缩成适合智能体读取的自然语言说明。

列摘要模块在生成列说明时输入列名、类型、基数、空值比例、数值范围、样例值和高频值，并要求输出稳定的业务语义，避免生成易过时的具体统计数字。对于表摘要，系统会聚合同表列信息和外键关系，描述表的职责、粒度和常见关联路径。对于文本文件，系统会提取文本分片摘要，使非结构化说明材料也能进入同一个检索空间。

语义向量模块只对存在非空 detail 的节点生成 embedding，并记录模型名、向量维度和 detail 文本 hash。当 detail 未变化且 embedding 配置一致时，模块会跳过该节点，避免重复请求 embedding 服务。向量写回 Neo4j 后，系统按节点标签创建向量索引，使 find(ref, query) 从名称匹配扩展为基于自然语言语义的实体检索。例如，当问题涉及“percentage”“ratio”“foreign player”或“school SAT score”时，系统可以召回对应表列、知识规则或历史案例。Starmie 等数据发现工作利用列表示和高效向量索引提升数据湖中的语义检索效率 [34]；Pontis 在规模较小的项目知识图谱中采用相同方向的思想，使智能体更快聚焦到候选实体，减少全量浏览 schema 的成本。

4.6 增量刷新与成本控制

图谱构建的成本主要来自三部分：源扫描、数据统计和 LLM/embedding 调用。Pontis 对这三部分分别采用不同策略。源扫描通过数据源 fingerprint 和刷新缓存避免重复发布未变化的数据源；数据统计尽量使用单次扫描、近似基数和 top-k 结构压缩原始值；LLM 摘要和语义向量则通过缺失字段检查、文本 hash 和批量请求减少重复调用。

这种设计使 Pontis 能在预处理成本和在线查询成本之间进行明确取舍。预处理阶段预先投入计算成本生成统计、摘要和关系线索，可以减少智能体在线阶段的工具调用轮次、SQL 探索次数和输入 token。另一方面，并非所有项目都需要运行完整流水线：在轻量场景中可以只运行基础事实发布和列统计；在复杂 Text-to-SQL benchmark 中，则可以追加列重叠、AI 摘要、向量索引和智能体探索，以换取更强的 schema linking、value grounding 和 join-path 可靠性。

5 工具智能体与执行验证机制

第五章介绍在线问答阶段的工具智能体与执行验证机制。与第四章的离线知识构建不同，本章关注当一个具体自然语言问题到来时，智能体如何组装上下文、调用工具、执行 SQL 验证，并在护栏反馈下修正最终输出。

5.1 智能体循环

Pontis 智能体采用工具调用循环。每轮循环中，系统将用户问题、系统提示、工具定义和历史消息发送给 LLM；若 LLM 返回工具调用，Pontis 执行对应工具并把结果作为下一轮输入；若 LLM 返回最终文本，则暂时结束循环等待用户反馈。

该过程可以表示为：

$$A_t = \text{LLM}(Q, H_{t-1}, T), \quad (5-1)$$

其中 Q 表示用户问题， H_{t-1} 表示历史上下文和工具结果， T 表示可用工具集合， A_t 表示第 t 轮模型动作。若 A_t 是工具调用，则系统执行工具并更新 H_t ；若 A_t 是最终 SQL 或自然语言答案，则结束循环。

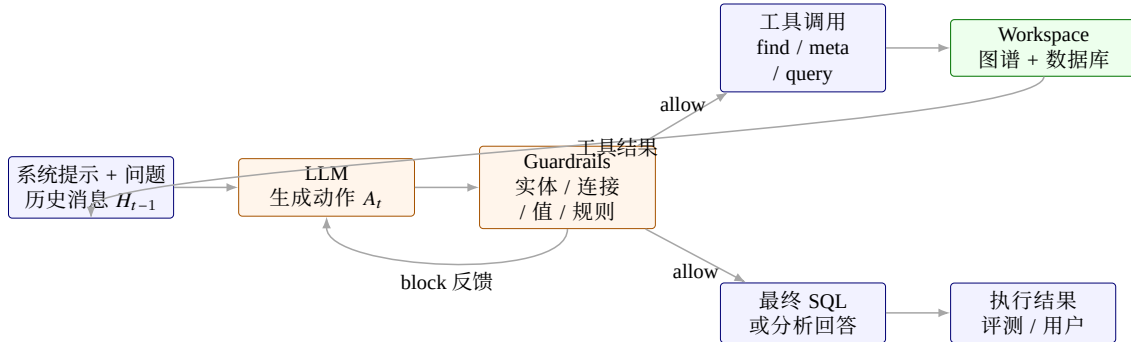


图 5-1 工具智能体与护栏执行流程

工具智能体的在线成本主要由 cached input token、uncached input token、output token 和 LLM 调用轮次共同决定。cached input token 对应可被 token 缓存命中的稳定前缀，uncached input token 对应每题运行时新增上下文，output token 对应模型生成内容，LLM 调用轮次则反映串行模型交互次数。

5.2 工具集合

Pontis 将工具划分为只读工具、写入工具和子智能体工具。通用只读配置包含实体查找、元数据读取、文件读取、JSON 下钻、内容搜索、SQL 查询、图查询和受限命

令执行等能力。写入模式增加知识实体创建、元数据更新、边添加和删除能力，用于经验沉淀和项目知识维护。需要强调的是，本文 BIRD 主实验并未开放全部只读工具，而是将在线求解工具限制为 `find`、`meta` 和 `query`，以减少任意文件读取、`shell` 探索和额外上下文对实验结果的影响。

表 5-1 Pontis 主要工具

工具	类别	作用
<code>find</code>	图检索	按 <code>ref</code> 模式和语义 <code>query</code> 查找实体
<code>meta</code>	元数据	查看实体属性和邻居关系
<code>query</code>	数据查询	在数据库、CSV/TSV、JSON 或工作空间范围内执行只读 SQL
<code>grep</code>	文本搜索	在文件实体范围内搜索内容
<code>read</code>	文件读取	读取文本文件指定行范围
<code>jd</code>	JSON 下钻	查看 JSON 文件结构和指定路径内容
<code>bash</code>	命令执行	执行受限 <code>shell</code> 命令
<code>cypher</code>	图查询	直接执行只读图查询
<code>create_entity</code>	图写入	创建知识实体
<code>update_meta</code>	图写入	更新实体元数据
<code>add_edge</code>	图写入	添加实体关系边
<code>delete</code>	图写入	删除实体
<code>agent</code>	子任务	调用子智能体完成局部任务

其中，`find`、`meta` 和 `query` 是 Text-to-SQL 场景中最核心的三个工具。`find` 用于缩小候选表列或知识范围，`meta` 用于精读实体语义和邻居关系，`query` 用于验证 SQL 假设和检查样例值。当前 `query` 工具不仅可以针对单个数据库文件执行只读 SQL，也可以把 CSV/TSV 或可表格化 JSON 映射为临时 SQLite 表；当 `ref` 为工作空间时，工具会注册项目中的数据库、表格和 JSON 表，并给出可用表名提示，从而支持更一般的数据项目问答。

5.3 提示组装

Pontis 的提示词并非单一固定 `prompt`，而是由稳定系统提示和任务实例提示共同构成。系统提示保留跨任务复用且对智能体理解数据项目必要的内容，包括基础身份、工具说明、图谱本体、SQL 规范、`effort` 约束、护栏规则、项目上下文和 README 摘要。

这种分层组装方式具有两个作用。第一，稳定系统提示可以在同一类任务中复用，减少重复 `token` 并有利于上下文缓存；第二，任务实例提示保留具体场景的约束，避免将评测输出格式、预处理写图策略或反思复盘规则混入所有智能体实例。工具相关提示词由创建智能体时显式传入的工具集合控制。

源码中的真实组装顺序为 `base`、`tool`、`ontology`、`sql`、`effort`、`guardrail`、`project`、`readme`。前几段在同一工具和护栏配置下高度稳定，被放在前缀以提高缓存命中；项目上下文和 README 更容易随数据库变化，因此被放在后部。BIRD benchmark 额外传入独立的任务 `prompt`、`question`、`evidence` 和输出格式约束，作为每题运行时上下文进入对话历史。

5.4 护栏机制

Pontis 的护栏层位于 LLM 输出和工具执行之间。每个护栏独立检查待执行工具调用或最终文本输出，并给出允许、警告或阻断裁决。多个护栏的裁决按阻断高于警告、警告高于允许的优先级聚合。

表 5-2 Pontis 护栏机制

护栏	文件	作用
RoundLimit	round_limit.py	达到轮次上限后阻止继续调用工具
ExplorationCheck	exploration_check.py	阻止过宽泛的全图探索
ToolAbuse	tool_abuse.py	限制指定工具的过量使用
SQLEntityCheck	sql_check.py	检查 SQL 引用表列是否已读且存在
BridgeTableCheck	sql_join_check.py	检查多表连接是否遗漏桥接关系
SQLDisambigCheck	sql_disambig_check.py	提醒同名列和局部术语消歧
SQLValueGroundingCheck	sql_value_grounding_check.py	检查最终 SQL 中字面过滤值是否能在真实列值中命中
BirdReadmeFinalRecheck	bird_readme_final_recheck.py	在 BIRD 最终 SQL 输出前回灌规则说明并强制自检一次
MetaDisambigPrefetch	meta_disambig_prefetch.py	读取 meta 后异步预取邻接消歧和 hint 上下文

护栏的设计目标不是替代模型推理，而是在高风险节点提供结构化反馈。例如，模型若尝试输出最终 SQL，但其中引用的表列从未通过 meta 阅读，SQL 实体检查会阻断最终输出并要求模型先核实实体；若 join 中涉及可能需要桥接表的列对，桥接表检查会提示相关关系实体；若 SQL 中出现与消歧实体相关的术语，消歧检查会要求模型读取对应解释；若最终 SQL 中出现 = 或 LIKE 字面值过滤且在目标列中命中 0 行，value grounding 检查会把高频值作为反馈返回给模型。

不同运行模式启用的护栏集合并不完全相同。通用只读 agent 默认使用轮次限制、探索检查、工具滥用限制、SQL 实体检查、桥表检查和 SQL 消歧检查。BIRD 基准求解阶段在此基础上显式加入 SQLValueGroundingCheck 和 BirdReadmeFinalRecheck。后者在本文实验配置中的实现方式是：当模型准备输出最终 SQL 后，直接将 BIRD SQL 写法约束作为 release checklist 回灌一次；无论模型先前 SQL 是否看似可行，系统都会阻断一次最终输出，要求模型重新检查输出列、排序、聚合、DISTINCT、LIMIT 和 SQLite 写法是否符合 BIRD 约束。该机制引入了一次额外 LLM 调用，但能够把评测特有的输出契约集中在最终自检阶段，避免将 BIRD 特定规则混入所有通用 agent 的默认提示。

6 实验设计与结果分析

6.1 研究问题

本文实验与分析围绕以下问题展开：

- (1) Pontis 相比只给 shell/sqlite/file 工具的 Bash Agent，是否能够通过图谱化数据工作空间提升 Text-to-SQL 准确率和稳定性？
- (2) Pontis 在解题过程中产生的 LLM 调用轮次和 token 资源消耗具有什么特点？
- (3) Pontis 与流水线基线方法在系统形态、上下文复用和预处理成本结构上有什么差异？
- (4) 在数据库结构和内容发生变化时，图谱化工作空间相较传统 pipeline 方法具有什么增量更新潜力？

6.2 数据集与评测指标

本文采用 BIRD 开发集作为主要实验数据。BIRD dev 包含 11 个数据库和 1534 道自然语言问题，提供 evidence、标准 SQL 和难度标签，适合评估真实数据库问答能力。需要指出的是，BIRD 中部分题目具有明显的数据集标注特征，百分比、比例、输出列、排序和 COUNT DISTINCT 等口径可能存在多种业务上合理的解释。因此，本文把准确度作为必要但不充分的指标，并结合 token 资源消耗和错误案例解释系统行为。

主准确度指标为执行准确率（Execution Accuracy, EX），即预测 SQL 与 gold SQL 在同一数据库上执行后的结果集合是否一致。除准确度外，本文还记录以下实验指标：

表 6-1 实验指标

指标	含义
Accuracy	SQL 查询结果比对得到的执行准确率
cached_input_tokens/Q	每题可被 token cache 命中的稳定输入上下文
uncached_input_tokens/Q	每题运行时新增、难以跨题缓存的输入上下文
output_tokens/Q	每题模型输出 token，包括工具调用参数、解释文本和最终 SQL
LLM Rounds/Q	每题串行 LLM 调用轮次
Total Tokens/Q	每题总 token 消耗，包含 cached input、uncached input 和 output token

不同方法的成本不宜只用总 token 粗略比较。对于工具智能体而言，每一轮 LLM 调用都会重新发送一部分稳定上下文和一部分动态上下文。稳定上下文通常包括系统提示、工具说明、项目级 README、数据库级提示和输出格式约束；动态上下文则包括当前问题、检索结果、工具返回、SQL 执行结果和前几轮模型输出。两者的复用性

质和实际成本不同，如果全部合并为 input tokens，就无法判断一个方法的成本来自稳定 prompt 过长，还是来自运行时探索过多。

基于这一考虑，本文将输入 token 拆分为 cached input 和 uncached input 两类。前者表示可被 token cache 命中的稳定输入前缀，后者表示每题运行时新增、难以跨题复用的输入内容。与此同时，output tokens 单独记录模型生成的工具调用、分析文本和最终 SQL。最终，total tokens 只作为总量参考，而不是唯一成本指标。单题 token 消耗可近似分解为：

$$C_{\text{token}} = C_{\text{cached}} + C_{\text{uncached}} + C_{\text{output}}. \quad (6-1)$$

其中 C_{cached} 表示系统提示、工具定义、项目 README 等可被 token 缓存命中的稳定前缀； C_{uncached} 表示每题运行时产生的问题、检索结果、工具返回和历史消息； C_{output} 表示模型生成的工具调用、分析文本和最终 SQL。由于不同 API 提供商对缓存输入、非缓存输入和输出 token 的计费方式不同，本文不将三者的权重固定为模型无关常数，而使用如下价格参数形式表示折算成本：

$$C_{\text{price}} = p_{\text{cached}}C_{\text{cached}} + p_{\text{uncached}}C_{\text{uncached}} + p_{\text{output}}C_{\text{output}}. \quad (6-2)$$

其中 p_{cached} 、 p_{uncached} 和 p_{output} 分别表示某一具体 API 服务中三类 token 的单位价格。因此，本文的主表首先报告原始 token 与轮次指标；只有在具体成本测算表中，才选取 DeepSeek V4 Flash 的价格作为示例进行大致折算。该折算结果用于比较同一价格假设下的相对资源结构，不应理解为所有模型或服务商上的固定成本。

实际日志中，本文只将表 6-1 所列轮次、缓存输入、非缓存输入、输出和总 token 作为跨方法比较的核心字段。缓存统计不是简单把“系统提示词长度”全部算作 cached。Pontis 的运行时统计优先读取模型服务返回的 cache hit/cache miss 字段；若服务端未返回显式缓存字段，则退化为基于连续请求最长公共前缀的估计，并在日志中记录 cache_accounting_source。旧版字段 input_tokens、pre_input_tokens 和 runtime_input_tokens 仅作为兼容历史日志的辅助字段，不进入本文主表。

6.3 对比方法

主实验比较五个方法：

为保证公平性，所有方法使用统一的大模型配置，并在需要 embedding 的环节使用本地 embedding 模型。实验采用 protocol-level zero-shot 设置：各方法在 BIRD dev 评测过程中不显式使用 BIRD train 集中的 gold SQL 或题目答案。需要说明的是，这并不等价于完全 benchmark-agnostic 的 zero-shot。CHESS、DeepEye-SQL、Alpha-SQL 和 Pontis 都在提示词、输出格式、评测脚本或数据库预处理流程中对 BIRD 的任务形式做了一定适配，因此更准确的表述是 BIRD-adapted zero-shot。

Pontis 在主实验中的配置也按 BIRD 协议做了收缩：在线求解阶段仅开放 find、meta、query 三个工具，prompt 段包括基础身份、工具说明、图谱本体、SQL 规则、护

表 6-2 主实验对比方法

方法	目录	定位
Bash Agent	Bash-Agent	无图谱、无专用预处理的通用工具 agent
Pontis	Pontis	图谱工作空间与工具智能体
CHESS	CHESS	检索、schema selection、候选生成和 unit test pipeline
Alpha-SQL	Alpha-SQL	基于 MCTS 的 test-time search 方法
DeepEye-SQL	DeepEye-SQL	value retrieval、schema linking、generation、revision、selection pipeline

栏规则、项目上下文、README 和 effort 约束，护栏包括轮次限制、探索检查、SQL 实体检查、桥表检查、消歧检查、值命中检查和最终 BIRD README 自检。该配置与通用 Pontis agent 的完整只读工具集不同，目的是让 BIRD 实验主要检验图谱上下文、元数据读取和 SQL 验证能力，而不是依赖任意文件读取或 shell 探索。

6.4 主实验结果

表 6-3 给出了五种方法在 BIRD dev 全量评测上的准确率与在线资源消耗。Pontis 在 1534 道题中答对 1003 道，执行准确率为 65.38%，比 Bash Agent 高 5.34 个百分点，与 DeepEye-SQL 的 65.25% 基本持平。Alpha-SQL 和 CHESS 分别取得 63.36% 和 58.67%。

表 6-3 BIRD dev 主实验结果与在线资源消耗

方法	正确数	准确率	轮次/Q	Cached/Q	Uncached/Q	Output/Q
Pontis	1003	65.38%	11.446	156569.7	6651.9	3174.7
DeepEye-SQL	1001	65.25%	5.339	4253.7	12637.4	5430.2
Alpha-SQL	972	63.36%	12.040	33972.8	18444.5	14181.0
Bash Agent	921	60.04%	6.746	6079.7	1544.8	1897.9
CHESS	900	58.67%	10.795	11191.5	15880.8	5142.9

从资源结构看，Pontis 的主要特点是 cached input 显著高于其他方法。这部分主要来自稳定系统提示、工具说明、项目级 README 和数据库上下文，能够在同一运行批次中被 token cache 命中；其 uncached input 为 6651.9 tokens/Q，低于 DeepEye-SQL、Alpha-SQL 和 CHESS。若只看 total tokens，容易高估 Pontis 的实际成本；在支持 prompt cache 的模型服务中，大量稳定前缀可以被折价复用，其实际成本结构与全部 uncached 的长上下文方法并不相同。Bash Agent 的成本最低，但准确率也明显低于 Pontis 和 DeepEye-SQL。Alpha-SQL 的输出 token 较高，反映了其多轮搜索和候选推理带来的生成成本。

表 6-4 给出了在线阶段和加入预处理摊销后的示例价格折算成本。这里不是定义通用固定倍率，而是以 DeepSeek V4 Flash 的缓存输入、非缓存输入和输出 token 价格为例，将三类 token 折算到同一成本尺度上。若不计 embedding token 的价格，Pontis 的预处理摊销后成本指数为 18709.1/Q；若将 embedding token 按输入 token 价格近似计入，

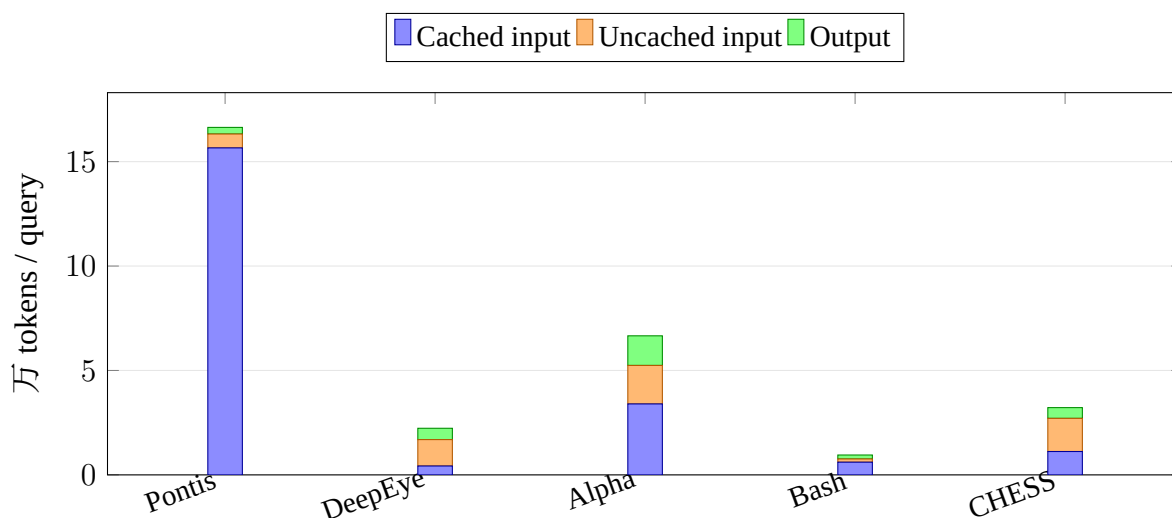


图 6-1 五种方法在线阶段 token 结构对比

则为 18820.0/Q。该结果说明，Pontis 的离线提取确实引入了预处理成本，但由于大部分在线上下文可缓存，在该价格假设下其摊销后成本仍低于 DeepEye-SQL、Alpha-SQL 和 CHESS。

表 6-4 五种方法的示例价格折算成本对比

方法	在线成本指数/Q	含预处理摊销/Q	含 embedding 摊销/Q
Bash Agent	5462.1	5462.1	5462.1
Pontis	16132.8	18709.1	18820.0
DeepEye-SQL	23582.9	23582.9	32475.5
CHESS	26390.4	26390.4	26398.0
Alpha-SQL	47485.9	48254.7	48281.4

6.5 预处理资源消耗

不同方法的预处理逻辑差异较大。Bash Agent 没有专门预处理；CHESS 主要构建轻量 embedding 索引；DeepEye-SQL 在预处理阶段主要生成 value-level 向量数据库；Alpha-SQL 除少量 embedding 外，还通过 LLM 生成部分数据库预处理资产；Pontis 则在图谱中写入统计、摘要、README、关系线索和向量索引。

Pontis 的预处理开销最高，主要原因是它把数据库理解过程前移到图谱构建阶段，而不是在每道题运行时重新探索。除在线问答成本外，本文同时报告预处理总成本和按 BIRD dev 查询数摊销后的平均成本。若忽略预处理成本，会低估图谱化工作空间的总体投入；若把预处理成本全部计入单题，又会低估长期项目中多次查询的摊销价值。该设计更适合一个数据库被长期、重复查询的场景；如果只对单个数据库提出极少量问题，预处理摊销不足，Pontis 的成本优势会减弱。

表 6-5 预处理阶段平均每库资源消耗

方法	LLM 轮次/库	Cached/库	Uncached/库	Output/库	Embedding/库
Pontis	219.0	1891828.4	120304.7	100566.7	15467.7
DeepEye-SQL	0.0	0.0	0.0	0.0	1240116.0
Alpha-SQL	139.5	87691.6	4994.6	50230.5	3732.6
Bash Agent	0.0	0.0	0.0	0.0	0.0
CHESS	0.0	0.0	0.0	0.0	1059.5

6.6 与流水线基线方法的差异

从系统形态看，CHESS、DeepEye-SQL 和 Alpha-SQL 更接近面向单次基准推理的流水线方法。它们通常将一次问题处理拆分为 schema linking、value retrieval、候选生成、执行验证、搜索或选择等阶段，目标是在当前 query 上最大化 SQL 正确率。Pontis 则把重点放在项目级工作空间上：预处理阶段将结构、统计、摘要、关系和消歧知识写入图谱，在线阶段的 agent 通过工具读取这些图实体并执行验证。

这种差异导致两类系统的成本结构不同。流水线方法的优势是对单题输入较轻，不一定需要维护长期图谱；但如果同一数据库被持续查询，许多 schema/value 探索会在不同问题之间重复发生。Pontis 的代价是前期预处理和较长的稳定上下文，收益则是这些知识可以被同一数据库下的后续问题复用，并且能够被检查、更新和归因。

从系统目标看，Pontis 与这些流水线基线方法的差异并不只是最终准确率高低。流水线方法更强调在单次 query 上完成尽可能充分的检索、搜索和选择；Pontis 更强调把可复用的数据库理解组织为长期工作空间，使后续查询可以复用并审计这些知识。因此，本文更关注 Pontis 在准确率接近最佳基线的时候，是否提供了更清晰的上下文组织、资源统计和增量维护结构。

6.7 数据库结构与内容变动下的增量更新潜力

本文讨论的增量更新不是 query 数量增加，而是数据库结构或内容发生变化后的系统维护问题。真实数据项目中，表可能新增或删除，列名和类型可能变化，外键约束可能补充，枚举值和数据分布可能漂移，说明文档和业务口径也可能更新。此时，Text-to-SQL 系统需要判断哪些上下文资产仍然有效，哪些索引、摘要、关系和经验需要刷新。

Bash Agent 没有持久化预处理资产，因此更新成本最低，但也无法复用数据库理解经验。CHESS、DeepEye-SQL 和 Alpha-SQL 可以通过重新构建检索索引、value vector DB、LSH 结构或预处理摘要来适配新数据库状态，但其更新粒度通常与数据库、表或预处理文件绑定，难以直接表达“某个列摘要过期但其他关系仍可用”的细粒度状态。

Pontis 的潜力在于把项目知识拆成具有稳定身份的图节点和关系。数据库、表、列、统计、摘要、关系、README、消歧规则和 embedding 都可以携带来源、时间戳、fingerprint 或文本 hash。当某张表新增列时，系统理论上只需刷新该表及其相邻列的结

构节点、统计、摘要和相关 embedding；当某个枚举列的值分布变化时，可以只标记该列统计、相关 value grounding 经验和依赖 README 片段为 stale；当外键或隐式连接发生变化时，可以沿关系节点传播 dirty 标记，重新运行局部 join 探索。与整库重跑相比，节点级和子图级刷新更适合长期运行的数据工作空间。

为了实现这种能力，Pontis 需要进一步完善六类机制：第一，源数据变更检测，包括 schema fingerprint、行数和列分布摘要；第二，派生节点的 provenance 记录，使每条摘要、关系或经验能够追溯依赖哪些源节点；第三，dirty flag 与依赖传播，用于判断哪些派生知识过期；第四，局部 extractor 调度，使系统只重跑受影响的统计、摘要、向量和 explorer；第五，stale-safe 查询模式，在在线回答时提示 agent 某些知识可能过期；第六，人工确认证据锁定，避免数据库短期噪声覆盖高置信度业务规则。

6.8 有效性威胁

本文实验存在若干有效性威胁。第一，所有方法虽然不显式使用 BIRD train gold SQL，但都不是完全 benchmark-agnostic 的 zero-shot。不同系统在 prompt、输出格式、BIRD evidence 使用方式和预处理脚本上存在适配程度差异，因此实验结论应理解为 BIRD-adapted zero-shot 协议下的系统比较。

第二，执行准确率受到 BIRD 标注噪声、query 歧义和输出契约的影响。后文第七章通过 Bash Agent 反思分类和若干样例展示这一问题，但本文没有对全部 1534 道题逐题人工复核，因此不能把所有错误都简单归因于系统缺陷或基准缺陷。

第三，本文没有进行完整消融实验，无法精确量化 Pontis 中图谱结构、列统计、README、语义向量、护栏机制和工具集合各自的独立贡献。现有结论主要来自与 Bash Agent 和流水线基线方法的整体系统比较。

第四，成本指标依赖当前模型、缓存策略、token 统计方式和运行脚本实现。本文使用统一日志字段比较 token 资源结构，并在具体测算表中以 DeepSeek V4 Flash 价格作示例折算；该折算结果不等同于固定服务商价格，也不保证在其他模型或缓存实现上保持相同排序。

第五，Pontis 与 DeepEye-SQL 的主准确率只相差 2 道题，差异较小，因此不能据此宣称 Pontis 在 BIRD dev 上显著优于 DeepEye-SQL。本文更合理的结论是：Pontis 在接近最佳基线准确率的同时，提供了图谱化、可缓存、可解释且具备增量更新潜力的系统结构。

7 基准局限与任务建模讨论

第三章已经从系统设计角度说明，本文关注的问题并不只是单次 SQL 生成，而是项目级数据理解。第六章进一步给出五种方法在 BIRD dev 上的准确率、资源消耗和系统形态差异。本章在此基础上讨论这些实验结果应如何被解释：当前 Text-to-SQL 研究的关键问题并不只是继续提升执行准确率，还包括一个更基础的问题，即现有静态基准是否真正刻画了真实业务中的自然语言数据分析任务。

因此，本章主要讨论当前 Text-to-SQL 评测本身的局限。BIRD 等数据集中的语义歧义、输出契约和标注口径问题说明，静态基准分数不能完全等同于真实业务正确性；如果只用一次性执行结果比对衡量系统，也难以反映真实数据分析过程中常见的业务澄清、口径协商和持续维护需求。

7.1 Text-to-SQL 任务建模局限

第三章已经将本文任务从传统 Text-to-SQL 扩展为项目级数据理解。本节进一步从评测解释角度说明，当前 Text-to-SQL 评测通常把任务抽象为从自然语言问题和数据库到 SQL 的映射函数：

$$f(query, database) \rightarrow sql. \quad (7-1)$$

但该建模隐含了一个较为严格的假设：给定同一个 query 和同一个 database，存在唯一正确的 SQL。实际情况往往并非如此。更准确地说，Text-to-SQL 不是严格的一对一函数映射，而更接近如下形式：

$$f(query, database, context) \rightarrow \{sql_1, sql_2, \dots\}. \quad (7-2)$$

其中 *context* 包括用户真实意图、业务口径、输出需求、历史查询习惯和数据集标注者偏好。造成这种非唯一性的原因主要有两类：一类是同一业务目标在 SQL 层面具有多重可达性；另一类是自然语言 query 本身具有语义欠定性。

这一点对本文实验解释具有重要意义。随着大语言模型能力提升，单纯 SQL 语法生成已经不再是最主要瓶颈。主流 agent 在语法层面较少失败，真正困难的问题往往是如何理解数据库结构、字段语义、过滤值、连接路径和业务目标。换言之，Text-to-SQL 正在从单纯的机器学习映射问题转向系统工程问题：系统需要组织上下文、调用工具、执行验证、处理歧义，并在必要时与用户澄清，而不只是直接拟合某条 gold SQL。

7.2 Bash Agent 反思结果的佐证

为了进一步说明静态执行准确率的解释边界，本文对 Bash Agent 在 BIRD dev 中的 613 道错误样本进行了反思式归因。该分析并不用于比较 Pontis、CHESS、Alpha-SQL 和 DeepEye-SQL 在每类错误上的表现，而是作为第七章讨论基准局限的经验材料：如果一个具备一定能力的通用工具 agent 的错误中有相当部分来自输出契约、隐含先验和问题欠定性，那么单纯把所有错误都归因于 SQL 生成能力不足并不充分。

表 7-1 Bash Agent 错误样本反思分类

类别	数量	占错误样本比例	含义
BIRD SQL 风格与输出契约	246	40.1%	SQL 可能在业务上合理,但输出列、排序、格式、聚合粒度或 DISTINCT/LIMIT 写法与 gold SQL 不一致。
数据集隐含先验	222	36.2%	仅凭 query 和数据库难以唯一确定 gold SQL 的表列选择、口径或标注偏好，需要额外数据集先验。
数据库探索可修复	145	23.7%	通过更充分的 schema、value、join 和样例验证，有机会修正的数据库理解错误。

表 7-1 显示，Bash Agent 错误样本中只有约四分之一属于相对直接的数据库探索可修复问题；更多样本落在 BIRD 输出契约和数据集隐含先验两类。这一结果并不意味着工具探索不重要，而是说明 Text-to-SQL 基准中的错误来源更加混合：有些错误可以通过更好的表列定位、值验证和连接路径搜索解决；有些错误则来自题面没有显式给出的标注者偏好、输出形状约束或业务粒度选择。后两类问题即使增加工具调用和模型推理，也不一定能稳定消除。

因此，Bash Agent 的反思分类可以作为本文核心判断的定量佐证：当前 Text-to-SQL 评测不只是考察模型能否生成语法正确的 SQL，也在考察模型是否复现某个静态数据集的标注口径。系统方法仍然有价值，特别是在数据库探索可修复类问题上；但当错误来自隐含先验或自然语言欠定时，更合理的系统形态可能是澄清、交互和业务目标确认，而不是要求模型在单轮预测中直接选择唯一 gold SQL。

7.3 SQL 多重可达性

第一，同一业务目标在 SQL 层面具有多重可达性。为了回答同一个业务问题，可能存在多种执行结果相同、语义相近，甚至对真实用户更有用的 SQL 写法。BIRD 数据集只选择其中一种作为 gold SQL，但这并不意味着该写法是唯一正确的。模型如果生成了另一种业务上合理的 SQL，仍可能因为输出列、排序、格式、数值类型或聚合粒度与 gold SQL 不一致而被判错。

在 card_games/q390 中，问题询问 ID 1-20 的卡牌颜色和格式。Bash Agent 返回了 id, colors, format，其中 id 有助于识别每张卡牌；但 gold SQL 只返回

colors, format, 因此额外输出 id 改变了执行结果形状并导致评测错误。类似地, 在 superhero/q763 中, 问题要求 Abomination 的 attribute value, Bash Agent 返回了 attribute_name, attribute_value, 对人类阅读更完整, 但 gold SQL 只接受 attribute_value。这类案例说明, 真实用户可能希望看到辅助列, 例如 id、名称、属性名或解释性字段; 基准则通常只接受 question/evidence 明确要求的最小输出列。

在 toxicology/q226 中, 问题要求五位小数百分比, Bash Agent 使用 `printf("%.5f", ...)` 输出固定五位小数字符串, 而 gold SQL 使用 `ROUND(..., 5)` 输出数值。两者在自然语言层面都可以解释为“保留五位小数”, 但执行结果比对会把字符串和数值视为不同结果。该例体现的是数值格式契约差异: 真实用户有时关心格式化后的展示结果, 基准则可能更偏好数值类型或某种 SQLite 函数写法。

这些例子反映了 BIRD 中常见的输出契约偏好。真实用户有时希望看到辅助列, 例如 id、名称、属性名或解释性字段; 基准则常常只接受 question/evidence 明确要求的最小输出列。真实用户有时关心格式化后的展示结果; 基准则可能更偏好数值类型或某种 SQLite 函数写法。模型生成的 SQL 如果在业务上更易读、更完整, 却没有严格复现 gold SQL 的输出契约, 仍会被执行结果比对判错。

这些例子说明, 基准准确率有时测量的是“是否匹配 gold SQL 的具体输出契约”, 而不完全等价于“是否回答了用户问题”。因此, 本文不应把 Text-to-SQL 简单视为“拟合唯一正确 SQL”的问题, 而应将其理解为一个带有语义欠定性和评测口径约束的系统工程问题。

7.4 Query 语义欠定性

第二类问题来自 query 本身的模糊性。自然语言问题通常是用户基于某种数据库先验写出的简短表达, 但这些先验并不会完整出现在 query 中。当数据库标注不够丰富、evidence 不够明确, 或者同一术语可以映射到多个表列时, agent 只能根据问题文本和数据库内容选择一个最可能的解释。这个解释在业务上可能合理, 却未必与 gold SQL 的标注选择一致。

在 codebase_community/q689 中, 问题要求找出“last to edit the post with ID 183”的用户。直觉上可以使用 postHistory 中最后一条编辑记录, 也可以使用 posts 表中的 last-editor 字段; 但 gold SQL 实际连接的是 owner 字段, 这与“last to edit”的自然语言表达并不完全一致。不同方法在这个题上选择了不同路径, 说明问题文本、数据库字段和标注 SQL 之间存在解释空间。

在 student_club/q1376 中, 问题询问 closed events 中 spend-to-budget ratio 最高的 event。一种自然解释是按 event 聚合预算行, 计算 $\text{SUM}(\text{spent})/\text{SUM}(\text{amount})$; 但 gold SQL 按单条 budget row 的 spent/amount 排序, 没有进行 event-level aggregation。两种写法对应不同业务粒度: 前者回答“某场活动整体预算使用率”, 后者回答“某条预算记录比例”。如果用户没有明确说明粒度, 系统难以仅凭 query 唯一确定哪一种是正确的口径。

在 thrombosis_prediction/q1187 中, 题面说 examined between 两个日期, 同时

条件来自 GPT 和 ALB。模型可能把 `examined date` 映射到 `Examination.Examination Date` 并连接 `Laboratory`，但 gold SQL 直接使用 `Laboratory.Date`。该例体现了日期字段来源歧义：两种写法都可以与 `examined` 语义建立联系，但它们过滤的是不同表中的日期字段，最终行集可能不同。自然语言中的 `examined date` 并不唯一指向某个表列，需要结合标注者口径才能确定。

这些案例说明，很多错误并不是模型不会写 SQL，而是 query、schema 和 gold SQL 之间存在欠定性。在真实系统中，更合理的处理方式可能不是直接猜测唯一 SQL，而是向用户澄清：“这里按 `event` 聚合还是按预算记录排序？”“`examined date` 指检查表日期还是实验室检测日期？”静态基准通常不允许这种交互，因此会把语义欠定问题压缩成单次预测错误。

7.5 执行结果比对的局限

现有评测通常采用两类指标：一类是 SQL 字符串或结构比对，另一类是执行结果比对。前者容易将语义等价但写法不同的 SQL 判为错误，因此已经逐渐被主流基准弃用；后者通过比较 predicted SQL 与 gold SQL 的执行结果，在一定程度上缓解了表面语法差异带来的假阴性。

但是，执行结果比对仍然没有解决前文提到的两个根本问题。首先，同一业务目标可能存在多种 SQL 实现路径，只要输出列、排序、聚合粒度或格式与 gold SQL 不一致，就可能被判错，即使该 SQL 在真实业务场景中是合理甚至更有用的。其次，自然语言 query 本身可能是欠定的，单句问题不足以唯一决定表选择、字段来源、过滤值和聚合口径。此时执行结果比对的衡量的仍然是“是否复现了标注者选择的那一种解释”，而不一定是“是否正确理解并满足了用户真实意图”。

更具体地说，执行结果比对会受到至少六类因素影响：输出列是否完全一致、排序是否一致、数值格式和类型是否一致、聚合粒度是否一致、是否使用 `DISTINCT` 或额外过滤条件、以及 gold SQL 本身是否存在标注偏好或错误。BIRD 相比早期基准确实更接近真实数据库场景，但它仍是静态基准。它能衡量模型与 gold SQL 的一致程度，却难以完整衡量模型是否理解了用户真正的业务目标。

这并不意味着执行准确率没有意义。相反，执行准确率仍然是跨方法比较中最稳定、最可复现的指标。问题在于，它应该被解释为“与基准标注的一致性”，而不是“真实业务正确性”的完整替代。本文在实验中报告准确率，同时引入 Bash Agent 反思分类、案例分析和资源消耗统计，正是为了避免把一个静态分数过度解释为系统能力的全部。

7.6 从静态评测到交互式业务目标

现有评测基准通常默认 Text-to-SQL 是一次性输入输出任务：用户给出一个自然语言 query，系统立即生成 SQL，并用 gold SQL 或其执行结果进行判定。但在真实数据分析场景中，用户问题经常存在歧义或信息缺失。为了得到可靠答案，系统往往需要

像 agent 一样进行追问、澄清和验证，例如确认时间范围、指标口径、输出粒度、实体定义或过滤条件。这意味着 Text-to-SQL 不应始终被建模为单轮映射问题，而应在复杂场景下被视为一个长序列的交互式任务。

在这种设定下，用户输入不再只是一个简短 query，而是逐步澄清后形成的明确业务目标。理想的系统流程更接近：

ambiguous query → *clarification/exploration* → *business goal* → *executable SQL*.
(7-3)

例如，当用户询问 ratio 时，系统可以确认是否乘以 100、是否保留小数、按行计算还是按实体聚合；当用户询问 latest 或 last edit 时，系统可以确认使用创建时间、更新时间、历史记录还是某个当前字段；当用户询问学校、球员或活动时，系统可以确认输出 id、名称还是完整解释列。这类澄清在真实业务中较为常见，但在静态基准中通常被禁止。

这也是本文将 Pontis 定位为项目级数据理解工作空间，而不是单纯面向静态基准优化的 SQL 生成器的原因。Pontis 的图谱、工具和护栏能够让 agent 查找相关表列、读取统计和样例值、执行 SQL 验证、记录消歧经验，并在未来系统中支持用户澄清。BIRD dev 实验验证的是该工作空间在静态协议下的效果；更长期的价值则在于它为交互式业务目标理解和持续知识维护提供了系统基础。

8 总结与展望

本文设计并实现了 Pontis：一种面向项目级数据理解的图谱化 Text-to-SQL 智能体工作空间。Pontis 不直接训练专用 SQL 生成模型，而是在原始数据和 LLM agent 之间引入可查询、可更新、可复用的图谱中间层，将数据库结构、列统计、值分布、语义摘要、关系线索、README 和经验知识组织为图实体及关系。基于这一工作空间，智能体可以通过 find、meta、query、grep、read、jd、cypher 和子智能体等工具逐步定位上下文，并通过护栏机制降低错误连接、未核实表列引用、无命中字面值和过度探索风险。

在 BIRD dev 全量 1534 道题上的 protocol-level zero-shot 实验中，Pontis 达到 65.38% 的执行准确率，高于 Bash Agent、CHESS 和 Alpha-SQL，并与 DeepEye-SQL 基本持平。资源消耗分析表明，Pontis 的在线阶段具有较高 cached input，但 uncached input 相对可控；在考虑预处理摊销后，若以 DeepSeek V4 Flash 价格作示例折算，其成本指数低于 DeepEye-SQL、Alpha-SQL 和 CHESS。Bash Agent 错误样本反思进一步说明，静态基准中相当一部分错误来自输出契约、隐含先验和语义欠定，而不只是数据库探索不足。

本文同时指出，Text-to-SQL 不应被简单看作从 query, database 到唯一 SQL 的函数映射。由于 SQL 实现路径多样、自然语言 query 欠定、gold SQL 存在口径偏好，执行准确率只能衡量系统与基准标注的一致程度，不能完全代表系统是否满足真实业务目标。因此，未来 Text-to-SQL 系统需要更多关注交互式澄清、项目级上下文维护、错误归因、资源成本和数据库变动下的增量更新能力。

后续工作可从三个方向展开。第一，补充系统化消融实验，分别验证图谱结构、列统计、README、语义向量和护栏机制对准确率及成本的贡献。第二，完善增量更新机制，建立源数据 fingerprint、派生知识 provenance、dirty flag 传播和局部 extractor 调度，使 Pontis 能在数据库结构和内容变化后按子图刷新。第三，探索更贴近真实业务的数据智能体评测方式，把单轮执行准确率扩展为面向业务目标的交互式评测和长期 query-log 式评测。

致 谢

参考文献

- [1] T Yu, R Zhang, K Yang, et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task[C] // Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. 2018 : 3911 – 3921.
- [2] J Li, B Hui, G Qu, et al. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs[J]. arXiv preprint arXiv:2305.03111, 2023.
- [3] F Lei, J Chen, Y Ye, et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows[J/OL]. arXiv preprint arXiv:2411.07763, 2025 [2026-05-21].
<http://arxiv.org/abs/2411.07763>.
- [4] G Sahu, A Puri, J Rodriguez, et al. InsightBench: Evaluating Business Analytics Agents Through Multi-Step Insight Generation[J/OL]. arXiv preprint arXiv:2407.06423, 2025 [2026-05-21].
<http://arxiv.org/abs/2407.06423>.
- [5] EPIC Data Lab, UC Berkeley, Hasura PromptQL. DAB: Data Agent Benchmark[J/OL]. Project website, 2026 [2026-05-21].
<https://ucbepic.github.io/DataAgentBench/>.
- [6] Microsoft. Fabric data agent concepts[J/OL]. Microsoft Learn, 2026 [2026-05-21].
<https://learn.microsoft.com/en-us/fabric/data-science/concept-data-agent>.
- [7] Snowflake. Cortex Agents[J/OL]. Snowflake Documentation, 2026 [2026-05-21].
<https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-agents>.
- [8] Databricks. What is a Genie Space[J/OL]. Databricks Documentation, 2026 [2026-05-21].
<https://docs.databricks.com/gcp/en/genie/index.html>.
- [9] Tableau. Agentic Analytics: A New Paradigm for Business Intelligence[J/OL]. Tableau Blog, 2025 [2026-05-21].
<https://www.tableau.com/blog/agentic-analytics-new-paradigm-for-business-intelligence>.
- [10] A Chen, M Bundele, G Ahlawat, et al. Text-to-SQL for Enterprise Data Analytics[J/OL]. arXiv preprint arXiv:2507.14372, 2025 [2026-05-21].
<http://arxiv.org/abs/2507.14372>.
- [11] J Johnson. QueryGPT - Natural Language to SQL using Generative AI[J/OL]. Uber Blog, 2024 [2026-05-21].
<https://www.uber.com/en-EG/blog/query-gpt/>.
- [12] K Maamari, F Abubaker, D Jaroslawicz, et al. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models[J/OL]. OpenReview preprint, 2024 [2026-05-21].
<https://openreview.net/forum?id=fglyh5pa7d#discussion>.
- [13] V Shkapenyuk, D Srivastava, T Johnson, et al. Automatic Metadata Extraction for Text-to-SQL[J/OL]. arXiv preprint arXiv:2505.19988, 2025 [2026-05-21].
<http://arxiv.org/abs/2505.19988>.
- [14] X Wang, M Costa, J Kovaceva, et al. Plugging Schema Graph into Multi-Table QA: A Human-Guided Framework for Reducing LLM Reliance[J/OL]. arXiv preprint arXiv:2506.04427, 2025 [2026-05-21].
<http://arxiv.org/abs/2506.04427>.

-
- [15] A Rissaki, I Fountalis, N Vasiloglou, et al. Towards Agentic Schema Refinement[J/OL]. arXiv preprint arXiv:2412.07786, 2024 [2026-05-21].
<http://arxiv.org/abs/2412.07786>.
- [16] R Sun, SO Arik, A Muzio, et al. SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL (extended)[J/OL]. arXiv preprint arXiv:2306.00739, 2024 [2026-05-21].
<http://arxiv.org/abs/2306.00739>.
- [17] S Talaei, M Pourreza, YC Chang, et al. CHESS: Contextual Harnessing for Efficient SQL Synthesis[J/OL]. arXiv preprint arXiv:2405.16755, 2024 [2026-05-21].
<http://arxiv.org/abs/2405.16755>.
- [18] M Deng, A Ramachandran, C Xu, et al. ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Consensus Enforcement, and Column Exploration[J/OL]. arXiv preprint arXiv:2502.00675, 2025 [2026-05-21].
<http://arxiv.org/abs/2502.00675>.
- [19] B Cao, W Liao, Y Sun, et al. APEX-SQL: Talking to the data via Agentic Exploration for Text-to-SQL[J/OL]. arXiv preprint arXiv:2602.16720, 2026 [2026-05-21].
<http://arxiv.org/abs/2602.16720>.
- [20] N Wretblad, F Riseby, R Biswas, et al. Understanding the Effects of Noise in Text-to-SQL: An Examination of the BIRD-Bench Benchmark[C/OL] // Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers). Bangkok, Thailand: Association for Computational Linguistics, 2024: 356–369 [2026-05-21].
<https://aclanthology.org/2024.acl-short.34/>.
- [21] T Jin, Y Choi, Y Zhu, et al. Pervasive Annotation Errors Break Text-to-SQL Benchmarks and Leaderboards[J/OL]. arXiv preprint arXiv:2601.08778, 2026 [2026-05-21].
<http://arxiv.org/abs/2601.08778>.
- [22] V Zhong, C Xiong, R Socher. Seq2SQL: Generating structured queries from natural language using reinforcement learning[C] // Proceedings of the 2017 ACM SIGMOD international conference on management of data. 2017: 236–247.
- [23] B Wang, R Shin, X Liu, et al. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers[C] // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020: 7567–7578.
- [24] Y Gao, H Liu, X Jia, et al. C3: Zero-shot text-to-SQL with ChatGPT[J]. arXiv preprint arXiv:2307.07306, 2024.
- [25] D Gao, H Wang, Y Li, et al. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation[J/OL]. arXiv.org, 2023 [2026-05-21].
<https://arxiv.org/abs/2308.15363v4>.
- [26] M Pourreza, D Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction[J]. Advances in Neural Information Processing Systems, 2024, 36.
- [27] PB Chen, F Wenz, Y Zhang, et al. BEAVER: An Enterprise Benchmark for Text-to-SQL[J/OL]. arXiv preprint arXiv:2409.02038, 2025 [2026-05-21].
<http://arxiv.org/abs/2409.02038>.
- [28] Z Chen, H Li, X Zhang, et al. RubikSQL: Lifelong Learning Agentic Knowledge Base as an Industrial NL2SQL System[J/OL]. arXiv preprint arXiv:2508.17590, 2025 [2026-05-21].
<http://arxiv.org/abs/2508.17590>.
- [29] X Wang, J Wei, D Schuurmans, et al. Self-Consistency Improves Chain of Thought Reasoning in

- Language Models[J/OL]. arXiv preprint arXiv:2203.11171, 2023 [2026-05-21].
<http://arxiv.org/abs/2203.11171>.
- [30] Z Abedjan, L Golab, F Naumann. Profiling Relational Data: A Survey[J/OL]. The VLDB Journal, 2015, 24(4): 557–581 [2026-05-21].
<https://doi.org/10.1007/s00778-015-0389-y>.
- [31] RC Fernandez, Z Abedjan, F Koko, et al. Aurum: A Data Discovery System[C/OL] // 2018 IEEE 34th International Conference on Data Engineering (ICDE). Paris, France : IEEE, 2018 : 1001–1012 [2026-05-21].
<https://doi.org/10.1109/ICDE.2018.00094>.
- [32] T Schick, J Dwivedi-Yu, R Dessi, et al. Toolformer: Language models can teach themselves to use tools[J]. Advances in Neural Information Processing Systems, 2023, 36 : 68639–68651.
- [33] S Yao, J Zhao, D Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C] // International Conference on Learning Representations. 2023.
- [34] G Fan, J Wang, Y Li, et al. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning[J/OL]. Proceedings of the VLDB Endowment, 2023, 16(7): 1726–1739 [2026-05-21].
<https://doi.org/10.14778/3587136.3587146>.
- [35] M Hulsebos, K Hu, M Bakker, et al. Sherlock: A Deep Learning Approach to Semantic Data Type Detection[C/OL] // Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. Anchorage, AK, USA : ACM, 2019 : 1500–1508 [2026-05-21].
<https://doi.org/10.1145/3292500.3330993>.

附录 A Pontis 代码模块说明

Pontis 的主要代码结构如下：

```
Pontis/
├─ storage/           # Workspace、Neo4j 图接口和发布触发机制
├─ extractor/         # 自动提取模块和流水线执行器
├─ explorer/          # agent 预处理模块
├─ tool/              # agent-facing 工具实现
├─ agent/             # LLM 循环、prompt、工具注册、guardrails
├─ utils/             # token、embedding 和通用运行统计辅助函数
├─ scripts/BIRD/      # BIRD 抽取和 benchmark 脚本
├─ docs/              # 架构、性能和实验分析文档
├─ pontis_cli.py      # 命令行入口
└─ pontis.yml         # 项目与 Neo4j 图配置
```

本文涉及的主要工具和模块可通过以下命令查看：

```
python -m extractor list
pontis <project_path>
python -m scripts.BIRD.extract --help
python -m scripts.BIRD.run_bird_benchmark --db card_games --limit 5
```